#1 Method `delete` takes a node and a point to delete from the node's subtree. It is defined recursively, and returns a pair of values: the first one tells if a point has been deleted in current subtree, and second one is `true` if current node now violates Ss-tree's invariants. We assume both `node` and `target` are non-null.

#2 If current node is a leaf, check that it does contain the point to delete and…

#3 … do remove the point…

#4 …and return `(true, true)` if now node contains less than m points, to let the caller know that it violates Ss-tree invariants and needs fixing, or `(true false)` otherwise, because anyway the point was deleted in this subtree.

#5 Otherwise, if this leaf doesn't contain `target`, we need to backtrack the search and traverse the next unexplored branch (the execution will return to line #7 of the call handling `node`'s parent – unless `node` is the root of the tree), but so far no change has been made, so it can return `(false, false)`.

#6 If `node` is not a leaf, we need to continue the tree traversal by exploring `node`'s branches. We start, though, by initializing a couple of temporary variables to keep track of the outcome of re cursive calls on `node`'s children.

#7/#8 Cycle through all of `node`'s children that intersects the target point to be deleted.

#9 Recursively traverse the next branch (one of the children that intersects `target`) searching for the point and trying to delete it.

#10 If the recursive call returns `true` for `violatesInvariants`, it means that the point has been